

Interview cheat sheet: all lessons

Every lesson's cheat sheet in one place. ★ = from the lesson; ♦ = my addition beyond the video.

01 · What Is Inference & Inference Engineering?

[Full notes](#)

Say it in 30 seconds

Inference is running a trained model to generate tokens, one at a time, for every request. Because it happens on every use, fleet-wide it costs more than training. Decode is limited by memory bandwidth, since every step streams all the weights to make one token per sequence. So the core trade is latency vs throughput vs cost, and batching is the first lever. The job is a loop: measure, find the bottleneck, change one thing, measure again.

Numbers to memorize

	Number	Why it matters
★	H100 HBM bandwidth 3.35 TB/s	Sets decode speed
♦	H100 BF16 dense ~989 TFLOPS	Sets prefill speed
♦	H100 ridge point ~300 FLOPs/byte	Below it you're memory-bound
★	Llama 3 8B in BF16 = 16 GB	Read in full on every decode step
♦	Llama 3 8B KV cache ~128 KB/token	Caps how many users fit in a batch
★	Batch-1 decode ceiling ~4.8 ms/step, ~200 tok/s	Best case for one user on one H100
★	\$4.17 vs \$0.28 per 1M tokens	Batching gives ~15× cheaper tokens
★	vLLM vs HF Transformers: up to 24× throughput	Serving software matters as much as hardware
★	DeepSeek: ~14.8k tok/s per 8-GPU node, 20–22 tok/s per user	High throughput and good latency together
★	Google ML energy: ~3/5 inference	"Train once, serve forever"

Formulas

	Quantity	Formula	Example (Llama 3 8B, H100)
★	Weight bytes	$\text{params} \times \text{bytes/param}$	$8\text{B} \times 2 = 16\text{ GB}$ (INT8: 8, INT4: ~4)
★	Decode step time (batch 1)	$\text{weight bytes} \div \text{bandwidth}$	$16\text{ GB} \div 3.35\text{ TB/s} \approx 4.8\text{ ms}$
★	Cost per 1M tokens	$\$/\text{GPU-h} \div (\text{tok/s} \times 3600) \times 10^6$	$\$3 \div (200 \times 3600) \times 10^6 \approx \4.17
♦	FLOPs per token	$\approx 2 \times \text{params}$	$\approx 16\text{ GFLOPs}$
♦	Arithmetic intensity (decode)	$\approx \text{batch size FLOPs/byte (ignoring KV)}$	batch 1 ≈ 1 , far below ~300
♦	Prefill time (ideal)	$\text{prompt} \times 2 \times \text{params} \div \text{peak FLOPS}$	$500 \times 16\text{ GF} \div 989\text{ TF} \approx 8\text{ ms}$
♦	KV bytes per token	$2 \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{bytes}$	$2 \times 32 \times 8 \times 128 \times 2 = 128\text{ KB}$

Prefill vs decode

	Prefill	Decode
Work	Whole prompt in one parallel pass	One token per sequence per step, serial
Bottleneck	Compute (FLOPs)	Memory bandwidth (bytes)
User sees it as	TTFT	Gap between tokens (ITL / TPOT)
Grows with	Prompt length	Output length, and batch × context (KV reads)

Symptom → lever (for "how would you fix it?" questions)

Symptom	Likely cause	First levers
High \$/token, GPU underused	Batch too small	★ Scheduling / continuous batching
Batch capped by out-of-memory	KV cache fills memory	★ Memory: pack KV tightly, reuse shared prefixes
Slow per-token speed at low load	Too many weight bytes per step	★ Precision: 8/4-bit weights
High TTFT	Long prompts or queueing	★ Shorter prompts, prefix reuse, ★ more replicas (fleet)
Model doesn't fit one GPU	Size	★ Parallelism across GPUs, ★ precision
Kernel time dominated by memory traffic	Unfused ops	★ Kernels: fuse operations
Any of the above		★ Off-GPU wins: smaller model, cached answers

Rapid-fire Q&A

Question	Crisp answer
Why is decode memory-bound?	Each step streams all the weights to make one token per sequence: ~1 FLOP/byte vs a ~300 ridge.
Why does batching help?	One weight read serves every sequence in the batch, raising FLOPs per byte.
What does batching cost?	Per-user latency rises (bigger steps, more KV reads) and KV memory grows.
◆ Max tok/s for 70B BF16 on 8×H100 (tensor parallel)?	140 GB ÷ 26.8 TB/s ≈ 5.2 ms/step, so ≤ ~190 tok/s. Real is lower (communication, KV).
Cost too high: where do you start?	Measure utilization, batch size, TTFT/ITL percentiles, then fix the biggest bottleneck first.
Will a faster GPU fix a slow service?	Not if the setup is bad: one request at a time just idles faster.

Traps to avoid

- Saying inference is "just `generate()`": it hides a scheduler, memory manager, kernels and server.
- Treating quality as tradeable: a faster but worse answer is not a win.
- Forgetting batching's cost: KV memory and per-user latency.
- Jumping to kernel tuning before checking model size, prompt length and caching.